System Hooks

A *hook* is a technique that lets you trap some Ruby event, such as object creation. Let's take a look at some common Ruby hook techniques.

Intercepting Method Calls

The simplest hook technique in Ruby is to intercept calls to methods in core classes. Perhaps you want to log all the operating system commands your program executes. You could simply use alias_method to rename the system method and replace it with a system method of your own that both logs the command and calls the original Kernel#system method, like this:

```
class Object
  alias_method :old_system, :system

  def system(*args)
    old_system(*args).tap do |result|
      puts "system(#{args.join(', ')}) returned #{result.inspect}"
    end
  end
end

system("date")
system("kangaroo", "-hop 10", "skippy")
```

produces:

```
Sun Dec 25 12:30:33 CST 2022
system(date) returned true
system(kangaroo, -hop 10, skippy) returned nil
```

However, the problem with this technique is that you're relying on there not being an existing method called old_system. A better alternative is to make use of method objects, which are effectively anonymous:

```
class Object
  old_system_method = instance_method(:system)

  define_method(:system) do |*args|
    old_system_method.bind_call(self, *args).tap do |result|
      puts "system(#{args.join(', ')}) returned #{result.inspect}"
    end
  end
end

system("date")
system("kangaroo", "-hop 10", "skippy")
```

produces:

```
Sun Dec 25 12:30:34 CST 2022
system(date) returned true
system(kangaroo, -hop 10, skippy) returned nil
```

You can also achieve similar behavior by using prepend to insert a module with a method of the same name earlier in the call chain. Within the module's methods, calling super calls the host's method of the same name. This gives us:

```
module SystemHook
```